

Blocco 3 - Normalizzazione e schema relazionale

Ridurre duplicazioni senza perdere significato

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Una tabella unica per vendite contiene cliente, ordine, prodotto, spedizione e ticket. Ogni ordine ripete dati cliente, ogni prodotto ripete categoria e prezzo, e una modifica a un dato anagrafico richiede aggiornamenti multipli.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Tabella unica delle vendite

Situazione concreta

Una tabella unica può contenere cliente, prodotto, ordine e spedizione nella stessa riga. All'inizio sembra comoda per produrre report.

ordine	cliente	email	sku	categoria
100	Sara	sara@mail	P10	sensori
101	Sara	sara@mail	P12	sensori
102	Marco	marco@mail	P10	sensori

Limiti della rappresentazione iniziale

- ▶ dati cliente duplicati;
- ▶ dati prodotto duplicati;
- ▶ anomalie di aggiornamento;
- ▶ impossibile registrare un prodotto mai venduto senza inventare un ordine;
- ▶ cancellando l'unico ordine si possono perdere dati anagrafici.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Da tabella piatta a schema normalizzato

Principio

La soluzione è separare le informazioni in base alle dipendenze: ciò che descrive il cliente va nella tabella clienti, ciò che descrive il prodotto va nella tabella prodotti, ciò che descrive la vendita va nelle tabelle ordine e righe ordine.

Esempio di riorganizzazione

- ▶ `customers(customer_id, email, name)`
- ▶ `products(product_id, sku, category)`
- ▶ `orders(order_id, customer_id, order_date)`
- ▶ `order_items(order_id, product_id, quantity, price)`

Processo di trasformazione

- ① individuare duplicazioni ricorrenti;
- ② riconoscere dipendenze funzionali;
- ③ separare fatti con identità autonoma;
- ④ conservare chiavi e riferimenti;
- ⑤ verificare che il report iniziale sia ricostruibile.

Analogia: indirizzo scritto su ogni pacco

Analogia o caso esplicativo

Se l'indirizzo del cliente viene copiato su ogni pacco, una correzione richiede trovare tutti i pacchi. Se invece esiste un'anagrafica cliente, il dato stabile vive in un solo posto; i pacchi conservano il riferimento e gli eventuali dati storici necessari.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

- ▶ dipendenza funzionale;
- ▶ anomalia di inserimento;
- ▶ anomalia di aggiornamento;
- ▶ anomalia di cancellazione;
- ▶ forme normali come criteri di progettazione.

- ▶ **Anomalia di aggiornamento:** la stessa informazione va modificata in più punti.
- ▶ **Anomalia di inserimento:** non posso registrare un fatto senza un altro fatto non ancora disponibile.
- ▶ **Anomalia di cancellazione:** cancellando un fatto ne perdo un altro.
- ▶ **Dipendenza funzionale:** un attributo determina un altro attributo.
- ▶ **Forma normale:** criterio per ridurre dipendenze sbagliate.

Come riconoscere una buona soluzione

- ▶ ogni relazione rappresenta un solo tipo di fatto;
- ▶ gli attributi descrittivi dipendono dalla chiave corretta;
- ▶ il report originale è ricostruibile tramite join;
- ▶ eventuali denormalizzazioni sono motivate da un uso concreto.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Nella tabella unica `sales_flat`, `customer_email` determina `customer_name`; `sku` determina `product_name` e `category`; `order_id` determina `order_date`.

Esempio: ragionamento

- ▶ i dati cliente non dipendono dalla riga ordine;
- ▶ i dati prodotto non dipendono dall'ordine;
- ▶ il prezzo storico può dipendere dalla riga ordine, non dal listino corrente;
- ▶ la spedizione può essere separata se ha stati e date propri.

Esempio: scomposizione normalizzata

- ▶ `customers(customer_id, email, name, country)`
- ▶ `products(product_id, sku, product_name, category)`
- ▶ `orders(order_id, customer_id, order_date, status)`
- ▶ `order_items(order_id, product_id, quantity, unit_price)`
- ▶ `shipments(shipment_id, order_id, shipped_at, delivered_at)`

Errori frequenti da prevenire

- ▶ normalizzare nomi di colonne senza capire le dipendenze;
- ▶ separare tabelle solo per categorie estetiche;
- ▶ perdere attributi storici necessari, come prezzo al momento dell'ordine;
- ▶ scambiare una vista/report per una relazione primaria.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Il dataset iniziale arriva come tabella piatta perché così è stato esportato dal gestionale. Il compito è trasformarlo in uno schema robusto prima di scrivere SQL operativo.

Esercitazione: specifica corretta

- ▶ input: elenco colonne di una tabella vendite denormalizzata;
- ▶ output: schema normalizzato con relazioni, chiavi e riferimenti;
- ▶ vincolo: non perdere dati storici necessari alle analisi;
- ▶ vincolo: motivare ogni dipendenza funzionale usata per scomporre.

Esercitazione: definizione della soluzione

- ① individuare entità stabili: cliente, prodotto;
- ② individuare eventi: ordine, riga ordine, spedizione;
- ③ spostare attributi descrittivi nella relazione determinante;
- ④ lasciare nella riga ordine prezzo e sconto storici.

Implementazione completa: schema normalizzato

- ▶ `customers`: dati anagrafici e segmentazione cliente
- ▶ `products`: descrizione prodotto e categoria
- ▶ `orders`: testata ordine e stato
- ▶ `order_items`: dettaglio prodotto, quantità, prezzo storico, sconto
- ▶ `shipments`: stato logistico collegato all'ordine
- ▶ `support_tickets`: richieste cliente collegate opzionalmente a un ordine

Esercitazione: organizzazione dei 90 minuti

- ▶ 20 min anomalie nella tabella unica;
- ▶ 25 min dipendenze funzionali;
- ▶ 25 min scomposizione e chiavi esterne;
- ▶ 20 min verifica con casi limite.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ normalizzare significa spostare informazioni nel posto dove hanno identità propria;
- ▶ la forma normale non è un rito: serve a prevenire anomalie concrete;
- ▶ una denormalizzazione si decide dopo aver capito la normalizzazione.